

GitHub-Sicherungskonzept

Version 1.0 · Stand 02.09.2026 · Verantwortlich Reza Nazarian · Betrieb edutech · Klassifizierung Intern

Dieses Konzept beschreibt, wie der in GitHub gehaltene Quellcode der ILP gesichert und im Ernstfall zurückgesichert wird. Es ergänzt die ILP-Technologie-Dokumentation um die drei prüfrelevanten Punkte:

Sicherungsarchitektur, Sicherungsintervalle und Rücksicherungstest.

Ausgangslage: In GitHub liegen Quellcode, Build-Pipelines und Container-Images — **keine personenbezogenen Daten und keine Produktivdaten**. Ein GitHub-Ausfall unterbricht daher Entwicklung und Auslieferung, nicht den Lernbetrieb. Git ist zudem von Natur aus verteilt: Jede Arbeitsumgebung hält eine vollständige Kopie der Historie. Die hier beschriebene Sicherung schützt gegen die Fälle, die diese Verteilung nicht abdeckt — Löschung, Überschreiben der Historie, Kontokompromittierung und dauerhafter Plattformausfall.

1. Sicherungsarchitektur

WAS GESICHERT WIRD

Sicherungsobjekt	Inhalt	Verfahren
Git-Repositories	Vollständige Historie aller Repositories (Content Factory, Content Delivery, AI-Backend, Infrastruktur) inklusive aller Branches und Tags	Spiegel-Klon (<code>git clone --mirror</code>)
Projektmetadaten	Issues, Pull Requests mit Reviews, Releases, Wiki — die Nachweiskette zu Freigaben und Änderungen	Export über die GitHub-API als JSON
Konfiguration	Branch-Schutzregeln, Teams und Berechtigungen, Repository-Einstellungen	Export als versionierte Konfigurationsdatei
Container-Images	Alle produktiv eingesetzten Versionen aus der GitHub-Registry	Kopie in die eigene Registry der Betriebsumgebung
Build-Pipelines	GitHub-Actions-Workflows	Teil der Repositories, dadurch automatisch mitgesichert

WIE GESICHERT WIRD



Pull-Prinzip: Der Azure-Job holt die Daten. GitHub hat keinen Zugriff auf den Backup-Speicher — eine Kompromittierung dort erreicht die Sicherungen nicht.

Sicherungsarchitektur: Pull aus GitHub, verschlüsselte Ablage außerhalb von GitHub, Rücksicherung wahlweise nach GitHub oder auf einen eigenen Git-Server.

- **Pull statt Push:** Die Sicherung läuft als **zeitgesteuerter Azure Container Apps Job** in der Azure-Umgebung von edutech und holt die Daten aus GitHub. Der Lesezugriff auf GitHub erfolgt über eine GitHub App mit minimalen Rechten, deren Zugangsdaten in **1Password** liegen und über ein eigenes Service-Konto erst zur Laufzeit in den Job geladen werden — sie stehen weder im Job-Image noch in der Azure-Konfiguration. Der Schreibzugriff auf den Backup-Speicher erfolgt über eine **verwaltete Identität** (Managed Identity) ganz ohne hinterlegte Zugangsdaten. GitHub selbst kennt die Ablage nicht.

- **Getrennte Ablage:** Die Sicherungen liegen verschlüsselt (AES-256) in einem **Azure Blob Storage** mit eigener Berechtigungsvergabe und zeitbasierter Unveränderlichkeitsrichtlinie, sodass sie auch bei einem kompromittierten Konto innerhalb der Aufbewahrungsfrist weder verändert noch gelöscht werden können.
- **Unabhängige Wiederherstellbarkeit:** Ein Spiegel-Klon ist ein vollwertiges Git-Repository. Die Rücksicherung ist nicht auf GitHub angewiesen und funktioniert auch auf einem eigenen Git-Server.

BENACHRICHTIGUNG BEI FEHLGESCHLAGENER SICHERUNG

Jeder Lauf schreibt sein Ergebnis nach Azure Monitor. Zwei Regeln greifen ineinander, damit weder ein *fehlgeschlagener* noch ein *ausgebliebener* Lauf unbemerkt bleibt:

- **Fehlerregel:** Bricht der Job ab, löst eine Warnregel in Azure Monitor eine Aktionsgruppe aus, die den Betrieb von edutech per E-Mail und Teams benachrichtigt.
- **Ausbleiben-Regel (Totmannschalter):** Bleibt eine Erfolgsmeldung länger als 26 Stunden aus, wird ebenfalls alarmiert — das deckt den Fall ab, dass der Job gar nicht erst startet.
- Bleibt eine Meldung unbearbeitet, eskaliert sie nach 24 Stunden an den Verantwortlichen. Der Status der letzten Läufe ist im Betriebs-Dashboard sichtbar.

Bewusst nicht gesichert: In GitHub hinterlegte Actions-Secrets sind technisch nicht auslesbar und damit nicht sicherbar. Sie sind deshalb **nicht die führende Quelle:** Alle produktiven Zugangsdaten liegen in **1Password** als führendem Secret-Store und werden von dort neu gesetzt. Der Verlust der GitHub-Secrets führt zu keinem Datenverlust, sondern lediglich zu einem erneuten Einrichten der Pipelines.

2. Sicherungsintervalle und Aufbewahrung

Sicherungsobjekt	Intervall	Aufbewahrung	RPO	Begründung
Git-Repositories	Täglich 02:00 Uhr	30 tägliche, 12 monatliche Stände	24 Stunden	Faktisch deutlich besser, da jede Arbeitsumgebung eine aktuelle Vollkopie hält
Projektmetadaten	Täglich 02:30 Uhr	30 tägliche, 12 monatliche Stände	24 Stunden	Nachweiskette zu Reviews und Freigaben; ändert sich laufend
Konfiguration	Wöchentlich, sonntags	12 wöchentliche Stände	7 Tage	Ändert sich selten und bewusst
Container-Images	Bei jedem Release, zusätzlich wöchentlich	Alle produktiven Versionen der letzten 12 Monate	Kein Verlust	Ermöglicht Auslieferung auch bei GitHub-Ausfall

WIEDERHERSTELLUNGSZIELE

Szenario	Ziel (RTO)	Vorgehen
Versehentlich gelöscht Repository	2 Stunden	Spiegel-Klon in ein neues Repository schieben, Konfiguration aus dem Export neu setzen
Überschriebene Historie (Force-Push)	1 Stunde	Betroffene Branches aus dem letzten Stand wiederherstellen
Komprommittiertes Konto	4 Stunden	Zugänge sperren, Sicherungsstand vor dem Vorfall prüfen und zurückspielen, Secrets rotieren
Längerer GitHub-Ausfall	1 Arbeitstag	Eigenen Git-Server aus den Spiegeln aufsetzen; Auslieferung läuft über die vorgehaltenen Images weiter

Wichtig für die Bewertung: Keines dieser Szenarien betrifft den laufenden Lernbetrieb. Die ausgelieferten Anwendungen laufen unabhängig von GitHub weiter; betroffen sind ausschließlich Entwicklung und die Auslieferung neuer Versionen.

3. Rücksicherungstest

Eine Sicherung gilt erst als wirksam, wenn die Rücksicherung nachweislich funktioniert. Der Test wird **halbjährlich** (Frühjahr und Herbst) durchgeführt, zusätzlich nach wesentlichen Änderungen an der Sicherungsarchitektur.

ABLAUF

Nr.	Schritt	Erfolgskriterium
1	Zufällig ausgewählten Sicherungsstand aus dem Backup-Speicher entschlüsseln und entpacken	Sicherung ist lesbar und vollständig
2	Repository in eine separate Testumgebung zurückspielen (nicht in die produktive Organisation)	Rücksicherung läuft fehlerfrei durch
3	Vollständigkeit prüfen: Anzahl Branches und Tags sowie letzter Commit gegen den Stand in GitHub abgleichen	Übereinstimmung ohne Abweichung
4	Anwendung aus dem zurückgesicherten Stand bauen und die automatisierten Tests ausführen	Build erfolgreich, Testsuite grün
5	Stichprobe der Metadaten prüfen (ein Pull Request mit Review, ein Release)	Inhalte vorhanden und lesbar
6	Vorgehaltenes Container-Image in der Testumgebung ausrollen	Anwendung startet und ist bedienbar
7	Testumgebung zurückbauen	Keine Rückstände, keine offenen Zugänge

DOKUMENTATION UND NACHVERFOLGUNG

- Jeder Test wird protokolliert: Datum, ausführende Person, geprüfter Sicherungsstand, Ergebnis je Schritt, benötigte Gesamtdauer.
- Die benötigte Gesamtdauer wird gegen die Wiederherstellungsziele aus Abschnitt 2 gespiegelt; Überschreitungen führen zu einer Anpassung der Ziele oder des Verfahrens.
- Abweichungen werden als Maßnahme mit Verantwortlichem und Termin erfasst und im Folgetest gegengeprüft.
- Die Protokolle werden fünf Jahre aufbewahrt und stehen der Revision sowie der Group-IT zur Einsicht zur Verfügung.

NOCH FESTZULEGEN

- Bestätigung, ob der Backup-Speicher im Bereich von edutech verbleibt oder auf die Sicherungsinfrastruktur der Group-IT aufgesetzt wird.
- Benennung der verantwortlichen Person je Testdurchführung und Vertretungsregelung.
- Termin des ersten vollständigen Rücksicherungstests.